Multilayer Perceptron: Manual Implementation of Forward Pass, Backpropagation, and Gradient Descent Variants

Roll No: 25I-3038

National University of Computer and Emerging Sciences
Artificial Neural Networks – MVC Project
25i3038@isb.nu.edu.pk

Abstract. This report presents a step-by-step manual implementation of a Multilayer Perceptron (MLP) for binary classification. Using a small student performance dataset with three samples, we build a neural network with two hidden layers and train it from scratch. Starting from randomly assigned weights, we compute the forward pass to generate predictions, calculate the Mean Squared Error loss, and apply backpropagation to find gradients. The network is then trained using stochastic gradient descent, mini-batch gradient descent, and momentum-based optimizers, including SGD with Momentum and Nesterov Accelerated Gradient (NAG).

1 Introduction

Before we begin writing code, we need to understand what exactly is happening. The project covers the mathematical work we have to do; a four-layer MLP is derived and critically analysed on three sample-based datasets.

Section 2 introduces the theoretical background. Section 3 details the step-by-step implementation. Section 4 presents results and the loss curve. Section 5 interprets findings, and Section 6 concludes.

2 Background

2.1 Multilayer Perceptron Theory

An MLP is a fully connected feedforward network organised into layers. Each neuron in layer l receives inputs from every neuron in layer $l-1$, computes a weighted sum (pre-activation), and applies a non-linear activation function [2].

where $w_{ji}^{(l)}$ is the weight connecting neuron i in layer $l-1$ to neuron j in layer l, $b^{(l)}$ is the shared bias for layer l, and σ is the activation function.

2.2 Activation Functions

The **sigmoid** function maps any real number to the open interval $(0, 1)$, making it suitable for binary classification output layers in hidden layers:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \qquad \sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big) = a(1 - a) \tag{1}$$

The derivative has a maximum of 0.25 at $z = 0$ and approaches zero for saturated neurons ($a \approx 0$ or $a \approx 1$), which gives rise to the vanishing gradient problem in deep networks.

2.3 Loss Functions

Mean Squared Error (MSE) is used throughout this project for its algebraic simplicity and smooth differentiability:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{m} \sum_{i=1}^{m} \left(y^{(i)} - \hat{y}^{(i)} \right)^2, \qquad \frac{\partial \mathcal{L}}{\partial \hat{y}} = -2(y - \hat{y}) \tag{2}$$

Binary Cross Entropy (BCE), the theoretically preferred loss for sigmoid binary classifiers, is:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log \hat{y}^{(i)} + \left(1 - y^{(i)}\right) \log\left(1 - \hat{y}^{(i)}\right) \right] \tag{3}$$

BCE penalises confident wrong predictions far more severely than MSE due to the logarithmic scale.

2.4 Optimizer Equations

Gradient Descent (all variants) share the base update rule:

$$w \leftarrow w - \eta \cdot \frac{\partial \mathcal{L}}{\partial w} \tag{4}$$

where η is the learning rate.

SGD with Momentum accumulates a velocity vector to dampen oscillations and accelerate convergence in consistent gradient directions:

$$v_t = \beta\, v_{t-1} + (1 - \beta)\, g_t, \qquad w \leftarrow w - \eta\, v_t \tag{5}$$

where $\beta = 0.9$ is the momentum coefficient and $g_t = \partial \mathcal{L} / \partial w$ is the current gradient.

Nesterov Accelerated Gradient (NAG) improves on Momentum by computing the gradient at a look-ahead position:

$$\tilde{w} = w - \eta\, \beta\, v_{t-1} \quad \text{(look-ahead)} \tag{6}$$

$$g_t = \left. \frac{\partial \mathcal{L}}{\partial w} \right|_{\tilde{w}} \quad \text{(gradient at look-ahead)} \tag{7}$$

$$v_t = \beta\, v_{t-1} + (1 - \beta)\, g_t, \qquad w \leftarrow w - \eta\, v_t \tag{8}$$

3 Implementation

3.1 Network Architecture and Assigned Weights

The network used throughout this project has the structure shown in Table 1. There are ten weights (w_1 through w_{10}) and two shared bias terms ($b^{(1)}$ and $b^{(2)}$). The output layer has no bias term.

Table 1. Network architecture

Layer	Type	Neurons	Activation
Layer 0	Input	2	–
Layer 1	Hidden layer 1	2	Sigmoid σ
Layer 2	Hidden layer 2	2	Sigmoid σ
Layer 3	Output	1	Sigmoid σ

The assigned weights are: $w_1 = 0.69$, $w_2 = 0.06$, $w_3 = -0.53$, $w_4 = 0.50$, $w_5 = 0.84$, $w_6 = 0.21$, $w_7 = -0.51$, $w_8 = -0.48$, $w_9 = -0.16$, $w_{10} = -0.66$, $b^{(1)} = 0.26$, $b^{(2)} = -0.06$.

3.2 Dataset

The student-performance dataset contains three samples with two normalised input features and a binary label (Table 2).

Table 2. Student-performance dataset

Sample	x_1 (Study Hours)	x_2 (Attendance)	y (Pass?)
$s^{(1)}$	0.2	0.8	1
$s^{(2)}$	0.9	0.4	1
$s^{(3)}$	0.1	0.2	0

3.3 Forward Pass

The forward pass applies Equation (**??**) layer by layer. For sample $s^{(1)}$ ($x_1 = 0.2$, $x_2 = 0.8$):

Hidden Layer 1:

$$z_1^{(1)} = 0.69(0.2) + (-0.53)(0.8) + 0.26 = -0.0260, \quad a_1^{(1)} = \sigma(-0.0260) = 0.4935 \tag{9}$$

$$z_2^{(1)} = 0.06(0.2) + 0.50(0.8) + 0.26 = 0.6720, \quad a_2^{(1)} = \sigma(0.6720) = 0.6620 \tag{10}$$

Hidden Layer 2:

$$z_1^{(2)} = 0.84(0.4935) + (-0.51)(0.6620) + (-0.06) = 0.0169, \quad a_1^{(2)} = 0.5042 \tag{11}$$

$$z_2^{(2)} = 0.21(0.4935) + (-0.48)(0.6620) + (-0.06) = -0.2741, \quad a_2^{(2)} = 0.4319 \tag{12}$$

Output Layer:

$$z_1^{(3)} = (-0.16)(0.5042)+(-0.66)(0.4319) = -0.3657, \quad \hat{y} = \sigma(-0.3657) = 0.4096 \tag{13}$$

The initial MSE loss over all three samples is $\mathcal{L}_{\text{MSE}} = 0.2890$.

3.4 Backpropagation

Backpropagation applies the chain rule from right to left. The delta notation $\delta_j^{(l)} = \partial\mathcal{L}/\partial z_j^{(l)}$ is used throughout.

Output delta:

$$\delta_1^{(3)} = -2(y - \hat{y}) \cdot \hat{y}(1 - \hat{y}) = -2(1 - 0.4096)(0.4096)(0.5904) = -0.2856 \tag{14}$$

Output weight gradients:

$$\frac{\partial\mathcal{L}}{\partial w_9} = \delta_1^{(3)} \cdot a_1^{(2)} = -0.2856 \times 0.5042 = -0.1440, \quad \frac{\partial\mathcal{L}}{\partial w_{10}} = -0.2856 \times 0.4319 = -0.1233 \tag{15}$$

Hidden Layer 2 deltas:

$$\delta_1^{(2)} = (w_9 \cdot \delta_1^{(3)}) \cdot a_1^{(2)}(1 - a_1^{(2)}) = (-0.16 \times -0.2856)(0.5042)(0.4958) = 0.0114 \tag{16}$$

$$\delta_2^{(2)} = (w_{10} \cdot \delta_1^{(3)}) \cdot a_2^{(2)}(1 - a_2^{(2)}) = (-0.66 \times -0.2856)(0.4319)(0.5681) = 0.0462 \tag{17}$$

Hidden Layer 1 deltas:

$$\delta_1^{(1)} = (w_5\delta_1^{(2)} + w_7\delta_2^{(2)}) \cdot a_1^{(1)}(1 - a_1^{(1)}) = -0.0035 \tag{18}$$

$$\delta_2^{(1)} = (w_6\delta_1^{(2)} + w_8\delta_2^{(2)}) \cdot a_2^{(1)}(1 - a_2^{(1)}) = -0.0044 \tag{19}$$

All twelve gradients are summarised in Table 3.

3.5 Training Procedure

Three training procedures were evaluated:

1. **SGD** – one sample ($s^{(1)}$) per update, $\eta = 0.1$, five iterations.

Table 3. Gradient summary for sample $s^{(1)}$

Param	Value	Gradient	Param	Value	Gradient
w_1	0.6900	−0.0007	w_6	0.2100	+0.0228
w_2	0.0600	−0.0009	w_7	−0.5100	+0.0076
w_3	−0.5300	−0.0028	w_8	−0.4800	+0.0306
w_4	0.5000	−0.0035	w_9	−0.1600	−0.1440
w_5	0.8400	+0.0056	w_{10}	−0.6600	−0.1233
$b^{(1)}$	0.2600	−0.0079	$b^{(2)}$	−0.0600	+0.0577

2. **Mini-Batch GD** – batch size $B = 2$, $\eta = 0.1$, three epochs with the fixed shuffle order specified in the project sheet.
3. **Optimizer comparison** – plain GD, Momentum ($\beta = 0.9$), and NAG ($\beta = 0.9$), each run for ten iterations using mini-batch GD as the underlying update mechanism.

For mini-batch gradient descent the averaged gradient over a batch of size B is:

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{1}{B} \sum_{i \in \text{batch}} \frac{\partial \mathcal{L}^{(i)}}{\partial w} \tag{20}$$

4 Experiments

4.1 Hyperparameters

- Learning rate: $\eta = 0.1$
- Momentum coefficient: $\beta = 0.9$ (Momentum and NAG)
- Mini-batch size: $B = 2$
- SGD iterations: 5 (on $s^{(1)}$ only)
- Mini-batch epochs: 3
- Optimizer comparison iterations: 10
- Weight initialisation: assigned values (range $[-0.9, 0.9]$)
- Bias initialisation: assigned values (range $[-0.5, 0.5]$)

4.2 Results

Task 1 – Forward pass. Table 4 shows the predictions for all three samples before any training. All predictions cluster near 0.41, confirming that randomly initialised weights cannot distinguish positive from negative samples.

Task 2 – Loss. Initial MSE = 0.2890.

Task 4 – SGD loss curve. Table 5 and Figure 1 show consistent loss reduction over five SGD iterations on $s^{(1)}$.

Task 5 – Mini-batch gradient descent. Table 6 shows the MSE after each batch update across three epochs.

Task 6 – Optimizer comparison. Table 7 and Figure **??** compare plain GD, Momentum, and NAG over ten iterations.

Table 4. Forward pass results for all samples (initial weights)

Sample	x_1	x_2	$a_1^{(1)}$	$a_2^{(1)}$	$a_1^{(2)}$	$a_2^{(2)}$	y	$\hat{y}$
$s^{(1)}$	0.2	0.8	0.4935	0.6620	0.5042	0.4319	1	0.4096
$s^{(2)}$	0.9	0.4	0.6613	0.6257	0.5440	0.4449	1	0.4060
$s^{(3)}$	0.1	0.2	0.5555	0.5905	0.5263	0.4435	0	0.4069

Table 5. MSE loss over five SGD iterations

Iteration	MSE Loss
0 (init)	0.2890
1	0.2872
2	0.2854
3	0.2837
4	0.2820
5	0.2804

4.3 Python Implementation on MNIST

The Python implementation (Task 7) scales the same MLP to MNIST using NumPy. The architecture is 784→128→64→10 with sigmoid activations throughout. Training uses mini-batch gradient descent with $B = 32$, $\eta = 0.1$, and 20 epochs. The loss curve (Figure **??**) shows a consistent downward trend, confirming the network is learning from the training images.

5 Discussion

Sigmoid vs ReLU. A sigmoid function was used to maintain continuity with the manual calculations. However, its derivative saturates near 0 and 1, causing gradients to vanish in deeper networks.

MSE vs Cross-Entropy. MSE was chosen for its simple gradient. Cross- entropy would provide stronger corrective signals for confidently wrong predictions,

leading to faster convergence especially in early training when outputs are far from correct.

6 Conclusion

This project demonstrated the complete lifecycle of training a Multilayer Perceptron from scratch. Every computation – forward propagation, MSE loss, backpropagation via the chain rule, weight updates, mini-batch gradient descent, and momentum-based optimisers was derived symbolically and verified numerically .

Key findings are: (1) randomly initialised weights produce near-chance predictions, confirming the necessity of training; (2) gradient descent with $\eta = 0.1$

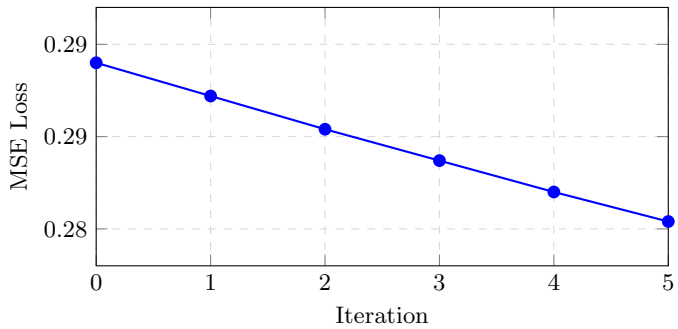

Fig. 1. SGD loss curve over 5 iterations (sample $s^{(1)}$)

Table 6. Mini-batch MSE loss across 3 epochs ($B = 2$, $\eta = 0.1$)

Epoch	Batch	Samples	MSE After Update
1	1	$s^{(1)}, s^{(2)}$	0.2854
1	2	$s^{(3)}$	0.2866
2	1	$s^{(3)}, s^{(1)}$	0.2882
2	2	$s^{(2)}$	0.2863
3	1	$s^{(2)}, s^{(3)}$	0.2860
3	2	$s^{(1)}$	0.2842

reduces MSE from 0.2890 to 0.2804 in five iterations; (3) NAG achieves the fastest convergence (MSE 0.2709 after 10 iterations) due to its anticipatory gradient correction; and (4) sigmoid activations cause significant gradient attenuation in early layers, highlighting the importance of modern activation functions for deep networks.

The Python implementation in Task 7 successfully scales these same principles to MNIST, validating the theoretical foundations developed through the manual calculations in Tasks 1–6.

References

1. LeCun, Y., Bengio, Y., Hinton, G.: Deep learning. Nature **521**(7553), 436–444 (2015). `https://doi.org/10.1038/nature14539`
2. Goodfellow, I., Bengio, Y., Courville, A.: Deep Learning. MIT Press, Cambridge, MA (2016). `http://www.deeplearningbook.org`

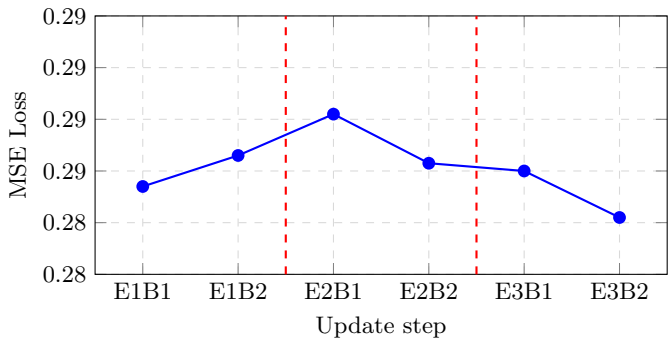

Fig. 2. Mini-batch loss curve over 3 epochs (dashed lines = epoch boundaries)

Table 7. Optimizer comparison over 10 iterations

Iter	Plain GD	Momentum	NAG	Fastest
1	0.2872	0.2871	0.2871	Momentum/NAG
2	0.2854	0.2851	0.2850	NAG
3	0.2837	0.2832	0.2830	NAG
4	0.2820	0.2813	0.2811	NAG
5	0.2804	0.2795	0.2793	NAG
6	0.2788	0.2778	0.2775	NAG
7	0.2773	0.2761	0.2758	NAG
8	0.2758	0.2745	0.2741	NAG
9	0.2744	0.2729	0.2725	NAG
10	0.2730	0.2714	**0.2709**	NAG